

QCL7350 – Quantum Inspired Optimization

Lab 1 – MaxCut and Knapsack Problems

Course Instructor: Prof. Kirankumar R. Hiremath, Dept. of Mathematics

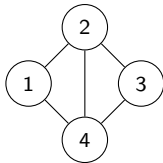
Teaching Assistant: Sumit Tapas Chongder, M.Tech Quantum Technologies,
IDRP-QIC, Dept. of Physics



1. MAXCUT PROBLEM (Unweighted)

Goal: Partition the vertices of an unweighted graph into two groups so that the number of edges between the groups (cut edges) is **maximized**.

Graph (unweighted)



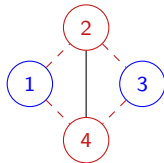
Edges: $\{(1, 2), (1, 4), (2, 3), (3, 4), (2, 4)\}$

Total edges $|E| = 5$

What is a cut edge?

An edge is a cut edge if its endpoints belong to **different** groups.

Example Optimal Partition



Group A (blue): $\{1, 3\}$ Group B (red): $\{2, 4\}$

Cut (broken edges): $(1, 2), (1, 4), (2, 3), (3, 4) \Rightarrow$ **Number of cut edges = 4**

Why is MaxCut hard?

- ▶ For n vertices, each vertex has 2 choices (which group it belongs to).
- ▶ Total binary assignments $= 2^n \rightarrow 2^{n-1}$ distinct partitions (ignoring group-label symmetry) $\rightarrow$ grows **exponentially!**

2. MAXCUT AS QUBO

We encode the partition using **binary variables** and convert the objective (maximize cut edges) into a Quadratic Unconstrained Binary Optimization (**QUBO**).

① Binary variables for vertices

For each vertex i , define $x_i \in \{0, 1\}$

$$x_i = \begin{cases} 0 & \text{if vertex } i \text{ is in Group A} \\ 1 & \text{if vertex } i \text{ is in Group B} \end{cases}$$

$$x_i = 0 \rightarrow \text{Group A} \quad x_i = 1 \rightarrow \text{Group B}$$

③ Objective: maximize number of cut edges

$$\text{MaxCut value} = \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$$

(Each cut edge contributes 1)

④ QUBO formulation (minimization)

In the minimization convention used here, the MaxCut objective is negated:

$$H_{QUBO} = - \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j) = \mathbf{x}^T \mathbf{Q} \mathbf{x} + c$$

② Cut indicator for an edge (i, j)

An edge (i, j) is cut if $x_i \neq x_j$.

Indicator: $x_i + x_j - 2x_i x_j$ equals 1 if edge is cut, else 0.

x_i	x_j	$x_i + x_j - 2x_i x_j$	Cut?
0	0	0	No
0	1	1	Yes
1	0	1	Yes
1	1	0	No

Minimizing H_{QUBO} is equivalent to maximizing the number of cut edges.

Solving this QUBO (classical or quantum) gives the partition with the **maximum cut**.

3. 0/1 KNAPSACK PROBLEM

Select items to **maximize total value** without exceeding capacity C .

Example (Capacity $C = 5$ kg)

Item	Weight (kg)	Value (Rs.)
A	2	20
B	3	30
C	4	40

Binary variables:

$x_i = 1 \rightarrow$ take item i

$x_i = 0 \rightarrow$ don't take item i

Objective (maximize):

$$20x_A + 30x_B + 40x_C$$

Constraint:

$$2x_A + 3x_B + 4x_C \leq 5$$

$$x_A, x_B, x_C \in \{0, 1\}$$

Brute Force:

► Try all $2^3 = 8$ combinations

► Best solution:

$$x_A = 1, x_B = 1, x_C = 0$$

$\rightarrow$ Value = 50, Weight = 5

Brute force grows as $2^n \rightarrow$
becomes **intractable!**

4. SOLVING KNAPSACK USING DYNAMIC PROGRAMMING ($O(nC)$)

DP builds a table $DP[i][w] = \text{max value using first } i \text{ items with capacity } w$ ($0 \leq w \leq C$).

Example ($C = 5$, items A,B,C)

Items	Capacity w (kg)					
	0	1	2	3	4	5
0 items	0	0	0	0	0	0
A (2,20)	0	0	20	20	20	20
B (3,30)	0	0	20	30	30	50
C (4,40)	0	0	20	30	40	50

Optimal value = 50 at capacity 5 $\rightarrow$ take A and B.

Complexity:

- ▶ DP time: $O(nC)$
- ▶ Here $n = 3, C = 5 \Rightarrow (n+1)(C+1) = 24$ table entries

Why faster?

- ▶ Brute force: $O(2^n)$
- ▶ DP: $O(nC)$ (big speed-up for moderate C)

5. KNAPSACK TO QUBO (PENALTY FORMULATION)

- ❶ Add slack variable for capacity:

$$2x_A + 3x_B + 4x_C + s = 5$$

Encode s in binary: $s = s_1 + 2s_2 + 4s_3$

$$\Rightarrow 2x_A + 3x_B + 4x_C + s_1 + 2s_2 + 4s_3 = 5$$

- ❷ We want to maximize value

$$20x_A + 30x_B + 40x_C$$

QUBO is **minimization** $\Rightarrow$ use negative value.

- ❸ Penalty for constraint violation:

Let $W(x, s) = 2x_A + 3x_B + 4x_C + s_1 + 2s_2 + 4s_3$

Hamiltonian (QUBO form):

$$H = -20x_A - 30x_B - 40x_C + \lambda[W(x, s) - 5]^2$$

Choose $\lambda > 0$ **sufficiently large** so that constraint violations are more costly than any possible value improvement.

- ❹ Expanding and using $x_i^2 = x_i$ (since $x_i \in \{0, 1\}$), the expression becomes $H = x^T Q x$ (QUBO).

For sufficiently large λ , minimizing H yields a feasible solution with maximum total value.